Can AI Foundation Models Beat Traditional Statistical Models?

Forecasting daily demand for 5,748 Walmart food items

Charlotte Zhu, Shihan Wang, Jing Chang, Rachel Liu

October 22, 2026

Table of contents

1 Executive summary **2**

2 Purpose and motivation **3**

3 Data **3**

4 Methodology and validation **4**
4.1 Models 4
4.2 Validation design 6

5 Results **7**
5.1 Overall accuracy: LightGBM leads the single models; the ensemble is best 7
5.2 Rankings change with the aggregation level 8
5.3 Claude: a general-purpose LLM as a forecaster 10
5.4 Computational efficiency 11

6 Business insight: the best order is not the forecast (the newsvendor problem) **12**

7 Conclusions **14**

8 Limitations and next steps **15**

References **16**

Appendix A: Interactive dashboard (Shelf Signal) **16**
A.1 Evidence from the original Shelf Signal 17

Appendix B: Use of AI tools and reproducibility **20**

1 Executive summary

Question. Can modern AI foundation models produce more accurate and computationally efficient retail demand forecasts than traditional statistical and machine-learning methods?

What we did. We forecast 28 days of daily unit sales for all 1,437 food items in Walmart's four California stores (5,748 item-store series, M5 data). Four main models were compared against a seasonal-naive benchmark: two classical time series models (**AutoETS** and **AutoARIMA**), the machine-learning method that dominated the M5 competition (**LightGBM**), and an AI foundation model for time series (Amazon's **Chronos**). We also tested a simple **ensemble**, the unweighted average of AutoETS and LightGBM. As an experiment, a general-purpose large language model (**Claude**) forecast a subset of series. No model saw the 28 test days while fitting.

What we found.

- **LightGBM is the most accurate single model across all series** (WRMSSE 0.646), with **AutoETS close behind** (0.661). AutoARIMA (0.892) and Chronos (0.949) trail the seasonal-naive benchmark (0.814).
- **Averaging the two best is better still.** The AutoETS + LightGBM ensemble scores 0.604, the best of all models and best at every aggregation level, with no tuning.
- **The ranking depends on the level.** LightGBM's lead is at store and department levels; for a single item in one store the models are close, with no model best on more than 31% of items. Close is not the same as statistically tied: we did not test these item-level gaps formally.
- **Claude matched, but did not beat, the best traditional approach.** On the 12 store × department totals it scored 0.579 (named) and 0.583 (blind), level with the ensemble (0.579) and ahead of every single model; on 100 sampled items it trailed AutoETS (0.959 vs 0.938). Hiding names and dates did not change its accuracy, but this cannot fully rule out that it has seen the public data.
- **Business insight.** The best order quantity is not the forecast: it is a quantile of the demand distribution set by the item's margin (the newsvendor problem). Turning forecasts into orders, ordering that quantile instead of the forecast was worth up to $172k over 28 days, far more than the choice of model; the more accurate model earned significantly more only at high margins.

Answer: not yet on accuracy, yes on speed. Traditional methods still lead on accuracy: LightGBM is the best single model, AutoETS is close, and their simple average is best overall. Chronos offers a large speed advantage; Claude matches the best traditional approach on totals but is slow and costly at scale, with too little evidence to recommend; and replenishment decisions depend on costs and uncertainty, not on forecast accuracy alone.

2 Purpose and motivation

Retail forecasting has to balance three things: accuracy, efficiency, and business value. LightGBM, a gradient-boosted decision-tree model, dominated the M5 forecasting competition, which asked teams to forecast daily Walmart sales for thousands of products; the organisers' analysis places it at the core of the leading entries (Makridakis, Spiliotis, and Assimakopoulos 2022). It became the default for retail demand forecasting.

Since then, two kinds of AI model have appeared that forecast without being trained on a retailer's data. **Time series foundation models**, such as Amazon's Chronos (Ansari et al. 2024), are pre-trained on millions of series and forecast new ones "zero-shot". **General-purpose large language models**, such as Claude, can read a sales history as text and reason their way to a forecast. If either can match LightGBM, a retailer could skip model training entirely.

We ask: **can modern AI foundation models produce more accurate and computationally efficient retail demand forecasts than traditional statistical and machine-learning methods?** Because a forecast is only useful through the decisions it drives, we close with a business insight on ordering (the newsvendor problem) and what it means for choosing a model.

3 Data

We use the M5 data (University of Nicosia and Walmart 2020) for the food category of Walmart's four California stores: 5,748 daily item-store series, with weekly prices, holidays and events, and SNAP (food-stamp) payment days.

Stores and items	CA_1 to CA_4; 1,437 food items in 3 departments
Unit of forecasting	One item in one store (item × store), 5,748 series
Training period	2011-01-29 to 2016-05-22 (1,941 days)
Test period	2016-05-23 to 2016-06-19 (28 days, the official M5 evaluation window)
Extra inputs	Prices, holidays and events, SNAP days

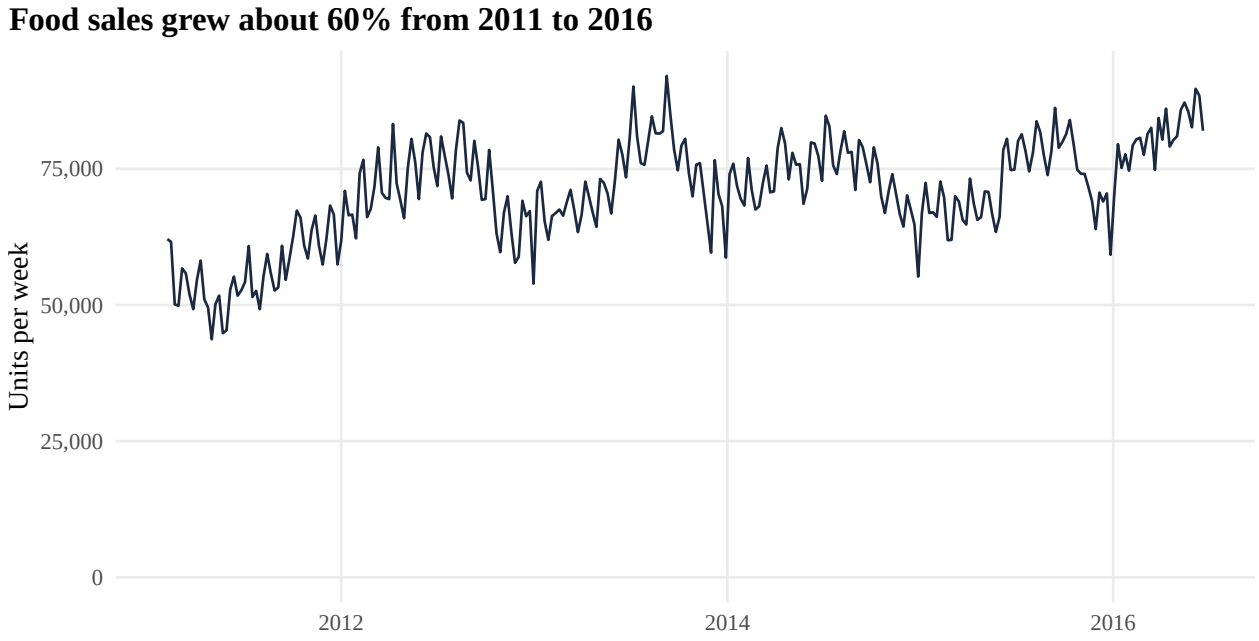


Figure 1: Weekly food units sold in the four California stores. The dip each December is the Christmas store closure.

Three features make the data hard to forecast. Weekends sell about 20% above an average day (Figure 2). Sales are 10% higher on SNAP days. And demand is sparse: the median item sold nothing on 51% of days in the last year.

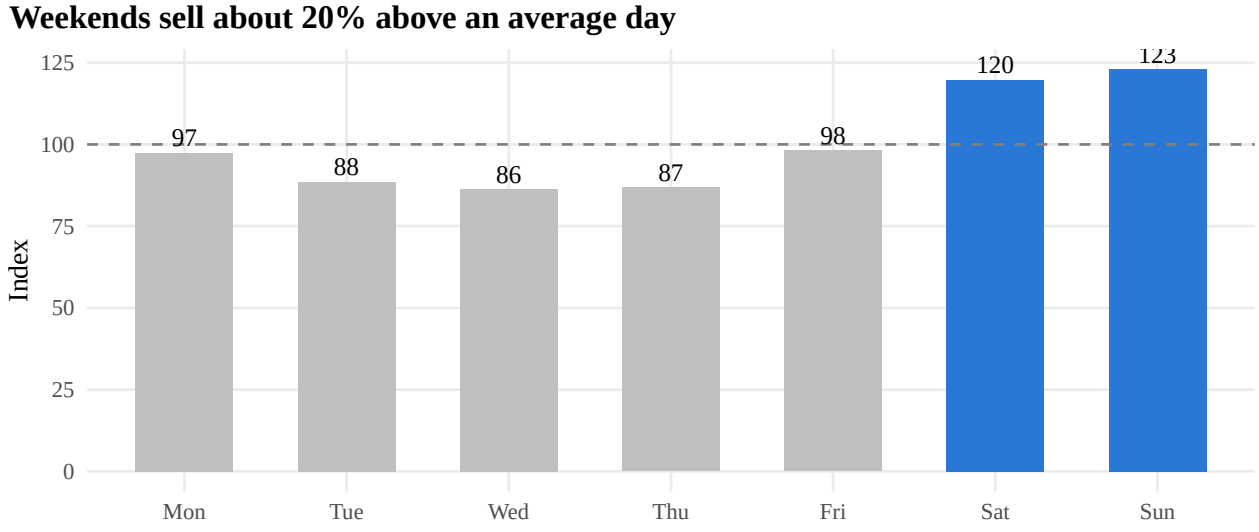


Figure 2: Average daily sales by weekday, indexed to the average day (= 100).

4 Methodology and validation

4.1 Models

Model	Family	Trained on our data?	Inputs	How it forecasts
AutoETS	Classical statistics	Yes, one model per series	Last 730 days of that series	Exponential smoothing with a 7-day season; error, trend and season form chosen automatically by AIC
AutoARIMA	Classical statistics	Yes, one model per series	Last 730 days of that series	Seasonal ARIMA with a 7-day cycle; orders chosen automatically by AIC
LightGBM	Machine learning	Yes, one global model	All series, plus price, weekday, month, holidays, SNAP	Gradient-boosted trees on sales lags of 28+ days and rolling means; Tweedie loss
Chronos-Bolt (small)	AI foundation model for time series	No (zero-shot)	Last 512 days of that series	Transformer pre-trained on about 100 billion observations; outputs quantiles
Ensemble	Combination	No extra fitting	AutoETS and LightGBM forecasts	Unweighted average of AutoETS and LightGBM
Seasonal naive	Benchmark	No	Last week	Repeats the same weekday from last week
Claude Opus 5.5 (experiment)	General-purpose large language model	No (zero-shot)	Last 365 days as text, with weekday, holiday, SNAP, price	Reads the numbers and reasons about them; no statistical model is fitted

Why these models. Each one stands for an approach a retailer could realistically use. AutoETS and AutoARIMA are the classical models covered in the course, fitted automatically so that no series needs hand-tuning. LightGBM is the machine-learning method that dominated the M5 competition, so it is the benchmark an AI model has to beat. Chronos-Bolt is a widely used, open time series foundation model that runs on a laptop; we chose the small version for speed. Claude represents general-purpose LLMs, the "ask ChatGPT or Claude" route. The ensemble tests the classic finding that combining a statistical and a machine-learning forecast helps, and seasonal naive is the floor every model must clear.

What information each model had. The models do not see the same inputs: AutoETS, Au-

toARIMA and Chronos use only the series' own history, while LightGBM and Claude also see prices and the calendar. None of them sees the test window. The Chronos paper keeps M5, and every retail dataset it uses, out of training as held-out test sets, so Chronos never trained on this data (Ansari et al. 2024).

How the AI forecasts were produced. *Chronos* reads the last 512 days of each series and returns nine quantiles (10th to 90th percentile) for each of the next 28 days; nothing is fitted. Its built-in point forecast is the median, which sits below the mean for sparse, right-skewed sales, so we use the average of the nine quantiles as its point forecast. *Claude* receives each series as text, one row per day, with an instruction to return 28 daily forecasts in a fixed JSON format with a short explanation; the code checks every reply and retries on a malformed one (details in Section 5.3).

4.2 Validation design

- **Out-of-sample test.** Every model is trained on data up to 2016-05-22 and forecasts the same 28 days. LightGBM's number of trees was tuned by early stopping on the last 28 *training* days, so the test window was never used for fitting or tuning.
- **Level of forecasting.** All main models forecast each of the 5,748 item × store series. Store, department and total forecasts are the sums of those item forecasts (bottom-up).
- **Accuracy metric.** The main metric is **WRMSSE**, the official M5 score: each series' root mean squared error is scaled by its in-sample naive one-step error and weighted by its dollar sales in the last 28 training days. Values below 1 beat a naive forecast. WRMSSE is averaged over six levels (total, store, department, store × department, item, item × store). We also report MAE, RMSE, the daily error (MAPE) of summed totals, and bias.
- **Claude subset.** Claude is called once per series through an API, so forecasting all 5,748 series would cost about $347 and take hours. It therefore forecast a subset of 112 series: 100 items sampled at random (fixed seed) from items with sales in the last 90 days, and the 12 store × department totals (4 stores × 3 food departments, FOODS_1 to FOODS_3; each total is the daily sum of every item in that department and store). Every other model is scored on exactly the same subset.
- **Direct vs summed totals.** Claude forecasts each total directly. For a fair comparison, AutoETS, AutoARIMA, Chronos and seasonal naive were also fitted directly on the 12 totals. LightGBM needs many series to train, so its totals remain sums of item forecasts.
- **Memorisation check.** Each Claude forecast was run twice: *named* (real dates, Walmart, item and store IDs, holiday names) and *blind* (no names or dates, only "day −364 … +28", weekday and a generic holiday flag).
- **Efficiency.** Each model was timed from prepared data to finished forecasts for all 5,748 series, including any training, one model at a time on the same laptop (10 CPU cores, Apple GPU). Data preparation and scoring, which are the same for every model, are excluded.

- **One test window.** All results come from a single 28-day window; see the limitations.

5 Results

5.1 Overall accuracy: LightGBM leads the single models; the ensemble is best

Table 3: Accuracy on all 5,748 series over the 28-day test window, sorted by WRMSSE.

Model	WRMSSE	MAE (units)	RMSE (units)	Daily error of total
Ensemble	0.604	1.518	3.028	3.6%
LightGBM	0.646	1.547	3.110	4.1%
AutoETS	0.661	1.548	3.157	4.9%
Seasonal naive	0.814	1.855	3.729	5.5%
AutoARIMA	0.892	1.563	3.226	9.9%
Chronos	0.949	1.523	3.387	10.4%

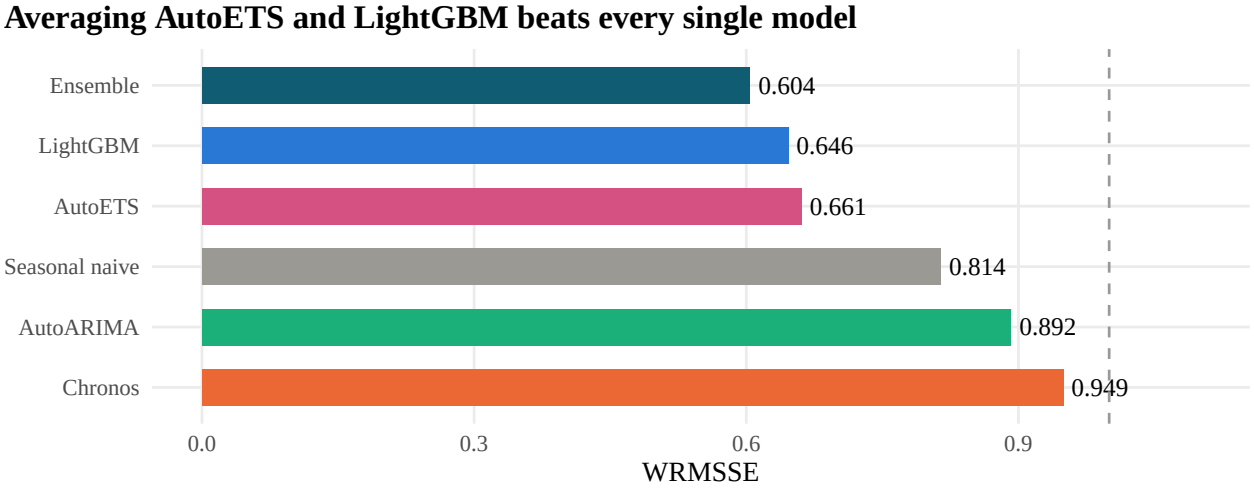


Figure 3: WRMSSE on all 5,748 series (lower is better). The dashed line at 1 is a naive one-step forecast.

Among single models, LightGBM has the lowest WRMSSE and RMSE, with AutoETS close behind; both beat the benchmark. Their unweighted average is better than either (0.604), echoing the M5 finding that combinations of forecasts are hard to beat (Makridakis, Spiliotis, and Assimakopoulos 2022). The two models make different kinds of errors, so averaging cancels part of them: summed over all items, LightGBM runs 1.0% low and AutoETS 1.4% high over the test window, and their average is within 0.2% of actual sales. Chronos has the lowest MAE of the single models but the weakest WRMSSE: MAE rewards getting the typical day right, while WRMSSE punishes errors that add up across items.

Summed over all items, LightGBM and AutoETS track the weekend peaks

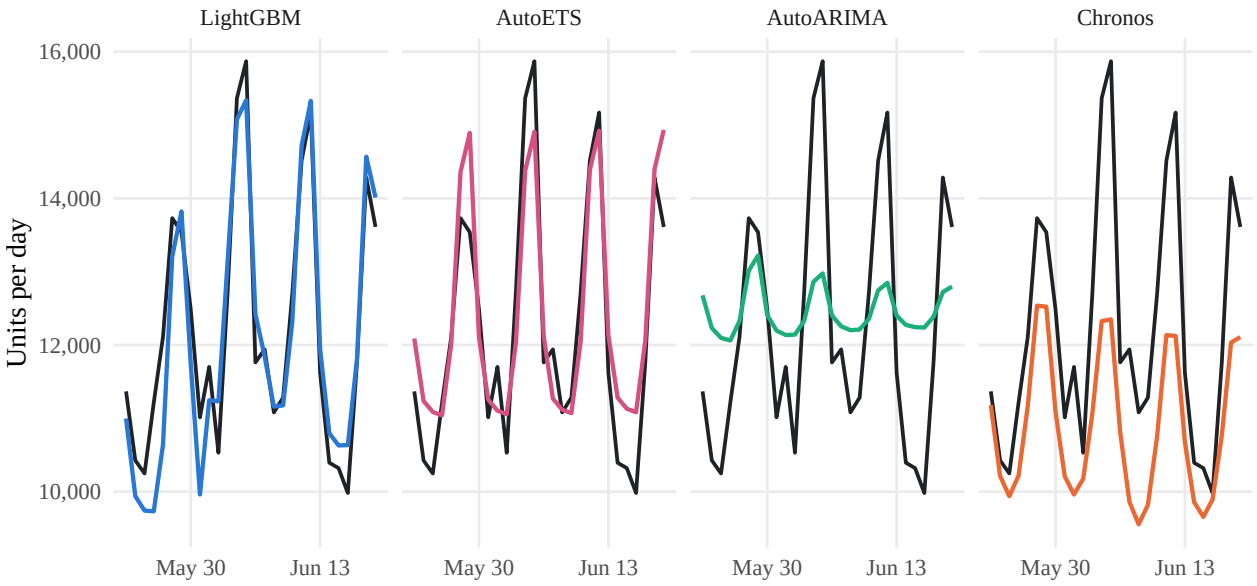


Figure 4: Total daily units across all items in the test window: actual (black) against each model's summed item forecasts.

5.2 Rankings change with the aggregation level

The models differ most at aggregate levels and converge at item level

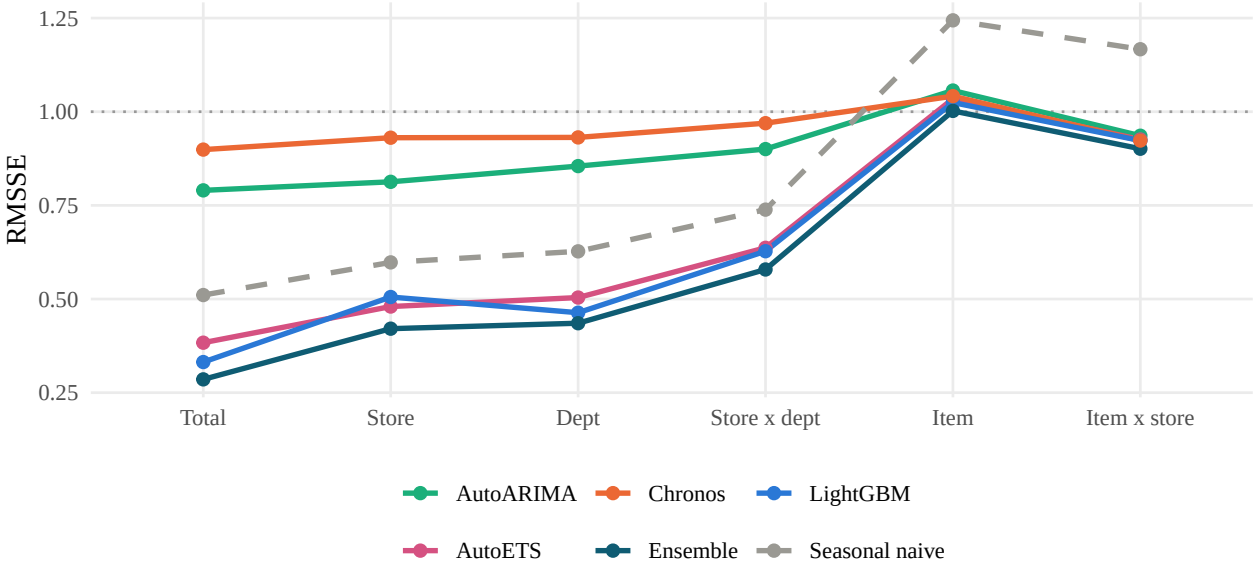


Figure 5: RMSSE by aggregation level (lower is better). Item-level forecasts are summed up to each level.

The models differ most at the levels retailers plan with: store, department and total. Summed to the four-store total, the daily error is 4.1% for LightGBM and 4.9% for AutoETS, against

9.9% for AutoARIMA and 10.4% for Chronos (Figure 4). The two weaker models fail in different ways. AutoARIMA produces nearly flat forecasts for items selling 0–2 units a day, and thousands of flat lines add up to a total that misses the weekly peaks. Chronos does follow each item's weekly pattern, so its total has the right shape, but it runs 11% low. The shortfall is not a trend effect: it grows with sparsity (Table 4). For an item that sells nothing on most days, the lower quantiles are near zero and the rare busy days lie above the 90th percentile, so the average of the nine quantiles misses them.

Table 4: Chronos's total forecast against actual sales over the test window, with items split into quarters by their sales in the last 56 training days.

Items	Actual units	Chronos forecast	Chronos vs actual
Sparsest quarter	23,559	11,990	-49%
Low	37,307	28,388	-24%
Middle	62,437	54,390	-13%
Busiest quarter	219,487	210,276	-4%

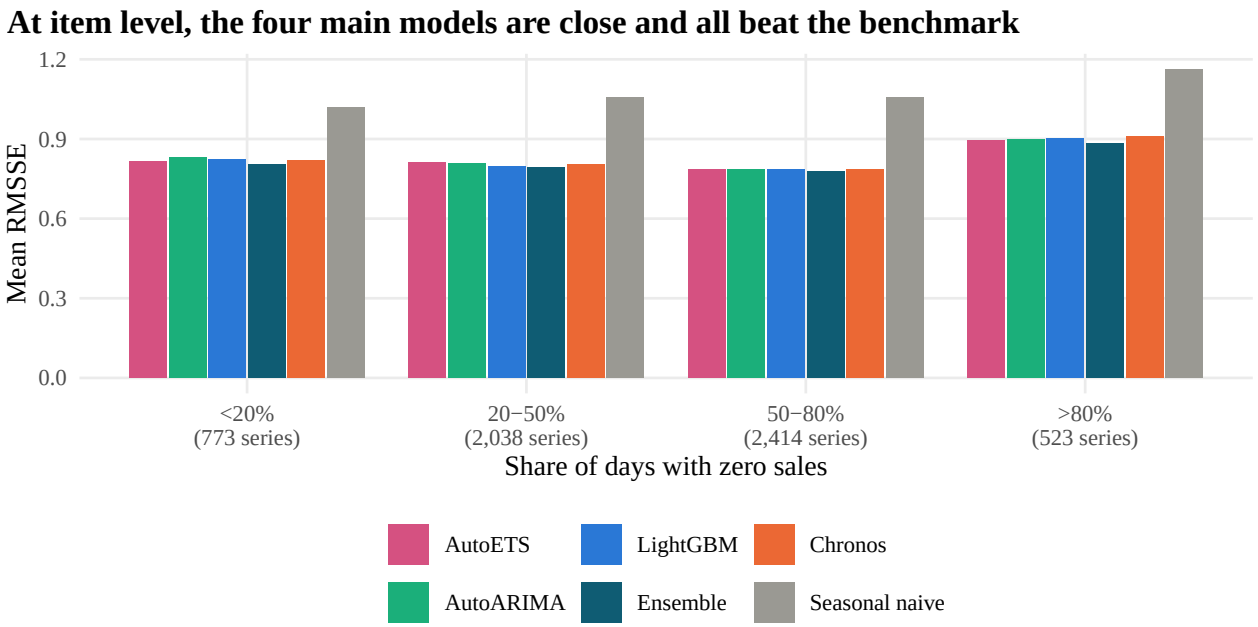


Figure 6: Mean RMSSE per item-store series, grouped by how often the item sells zero.

For one item in one store, the models are close: among the four single models, each is the most accurate on part of the items (LightGBM 31%, Chronos 27%, AutoETS 23%, AutoARIMA 19%), and all beat the benchmark by a similar margin. Daily sales of a single item are mostly noise. "Close" here is descriptive; we did not run a formal test of equal accuracy.

5.3 Claude: a general-purpose LLM as a forecaster

Claude received each series' last 365 days as text (date, weekday, holiday, SNAP day, price, units) and returned 28 daily forecasts in a fixed JSON format, together with a short explanation of its reasoning. It cost $13.54 for 224 forecasts.

On the 12 totals, Claude matches the AutoETS + LightGBM ensemble

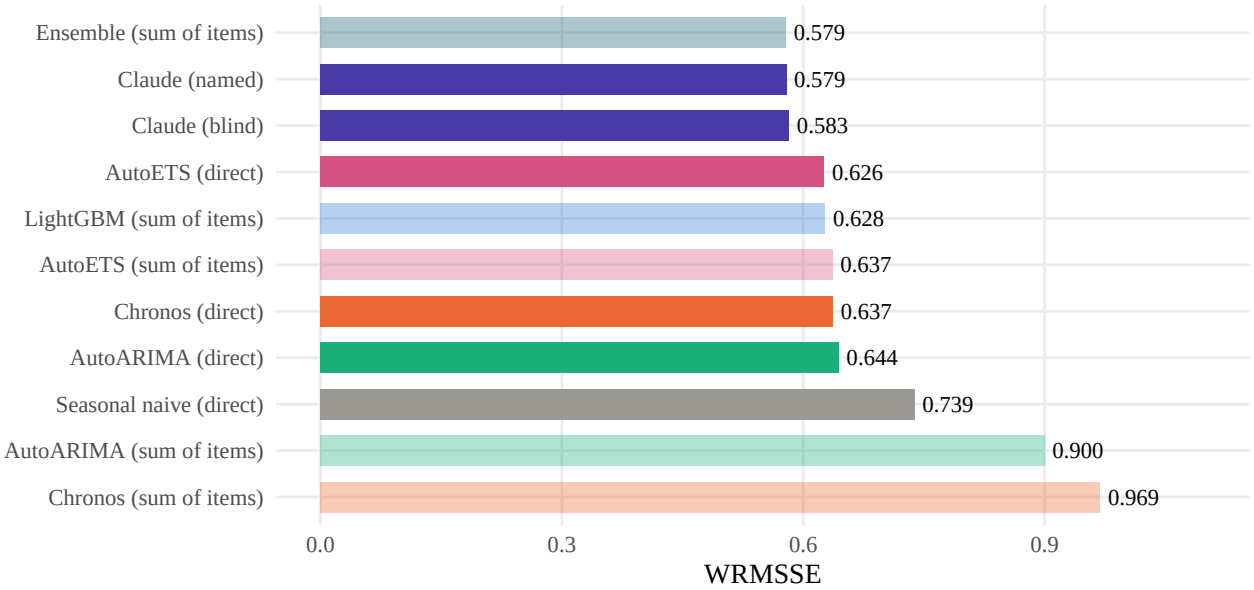


Figure 7: WRMSSE on the 12 store × department totals. Pale bars are totals built by summing item forecasts; solid bars are fitted directly on each total.

Table 5: Accuracy on the 100 sampled items (same items for every model).

Model	WRMSSE	MAE (units)
AutoETS	0.938	1.577
AutoARIMA	0.951	1.652
Claude (named)	0.958	1.674
Claude (blind)	0.959	1.665
Ensemble	0.962	1.662
Chronos	0.995	1.697
LightGBM	1.031	1.836
Seasonal naive	1.138	1.867

On the 12 totals, Claude (0.579 named, 0.583 blind) was level with the ensemble (0.579) and ahead of every single model, the best of which was AutoETS (direct) (0.626). Fitting the totals directly also matters: AutoARIMA improves from 0.900 to 0.644 and Chronos from 0.969 to

0.637. On the 100 items, Claude ranked behind AutoETS and AutoARIMA. Named and blind forecasts were equally accurate (0.579 vs 0.583 on totals), so real dates and product names gave Claude no advantage.

How Claude differs from Chronos. Both are transformer models used zero-shot, but they are different tools:

	Chronos	Claude
Trained on	Time series (M5 held out)	Text and code
Reads our data as	Scaled numbers	Text, one row per day
Inputs	Sales history	Sales, calendar, price
Output	Quantiles	Forecast + written reasoning
All 5,748 series	10 s, free	~3.7 h, ~$347
Same output on rerun?	Yes	Not exactly

Risks of using an LLM, and how we handled them.

Risk	What we did	What remains
Contamination (M5 is public)	Named vs blind test	Recalled patterns can't be ruled out
Unstable output	Fixed prompt and model; replies saved	Rerun variation not measured
Cost and scale	112-series subset, replies cached	May not hold for all 5,748
Malformed answers	Fixed JSON, 28-day check, retries	None in 224 calls

5.4 Computational efficiency

Table 8: Time to fit and forecast all 5,748 series on the same laptop. Claude is scaled up from its measured run.

Model	Time, all series	Training	Cost per run
Chronos	10 s	None	Free
LightGBM	80 s	About 1 min	Free
AutoETS	86 s	Per series	Free
AutoARIMA	39 min	Per series	Free
Ensemble	3 min	Both models	Free

Table 8: Time to fit and forecast all 5,748 series on the same laptop. Claude is scaled up from its measured run.

Model	Time, all series	Training	Cost per run
Claude	~3.7 h (est.)	None	~$347 (est.)

Chronos is about 232 times faster than AutoARIMA and 8 times faster than LightGBM, with no training. Its strength in this study is speed rather than accuracy. Claude is the opposite: accurate on the totals, but slow and costly at scale.

6 Business insight: the best order is not the forecast (the newsvendor problem)

A forecast is only valuable through the decisions it drives. For perishable food, each day the store orders Q units of an item with price p and gross margin m. A missed sale loses the margin, $c_u = pm$; an unsold unit wastes its cost, $c_o = p(1-m)$, assuming leftover food has no salvage value. Expected profit is maximised by ordering

$$Q^* = F^{-1}\left(\frac{c_u}{c_u + c_o}\right) = F^{-1}(m),$$

the m-th quantile of the demand distribution F. A 20%-margin item should be ordered at the 20th percentile of demand (P20), and an 80%-margin item at the 80th percentile (P80). **The optimal order is generally not the point forecast**: it requires a forecast of the whole demand distribution, not just its average.

This is where Chronos's output is useful. Point-forecast models (AutoETS, LightGBM, the ensemble) need an assumed distribution around their forecast to give a P80. Chronos forecasts the quantiles directly, so the order can be read straight off its output for margins between 10% and 90%. Its upper quantiles were well calibrated in the test window: actual sales were at or below its 80th percentile on 78% of item-days and at or below its 90th on 89%. The lower quantiles look over-covered (45% at the 10th percentile) mainly because many items sold zero and Chronos forecast near zero, which counts as covered. So a model that is weaker on point accuracy can still be useful for ordering, which is why the metric should match the decision.

To test this, we turned each model's forecasts into daily orders for every item over the 28 test days and scored them against actual sales, under two rules: **order the point forecast**, or **order the margin quantile** of a negative binomial distribution centred on the forecast. Margins run from 20% to 80%.

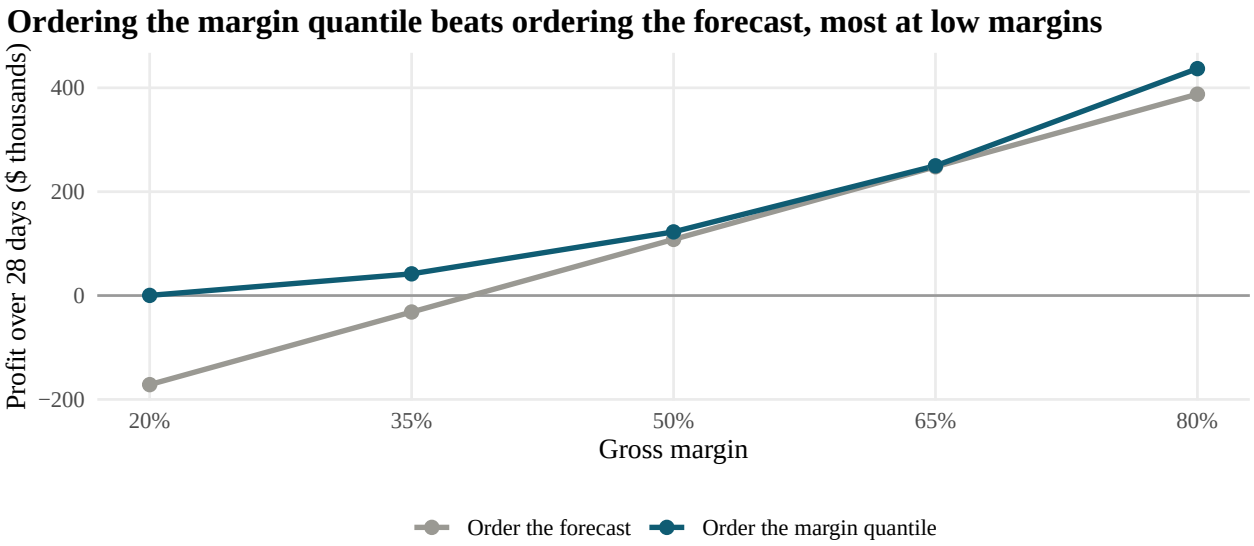


Figure 8: Profit over 28 days for all items when the AutoETS + LightGBM ensemble's forecasts become orders: ordering the forecast itself against ordering the margin-matched quantile. The other models' lines lie close to these.

The ordering rule matters far more than the model (Figure 8). At a 20% margin, ordering the ensemble's forecast loses $172k over 28 days, because every unsold unit wastes 80% of its price; ordering the 20th percentile instead roughly breaks even. At an 80% margin, the quantile rule earns $49k more by stocking deeper. The two rules nearly coincide around a 65% margin, where the forecast happens to sit close to the matching quantile. By comparison, under the quantile rule the four models never differ by more than $20k at any margin.

Did the more accurate model also earn more? To check whether profit gaps between models were real, we resampled the items 2,000 times (Figure 9; Chronos used its own quantiles).

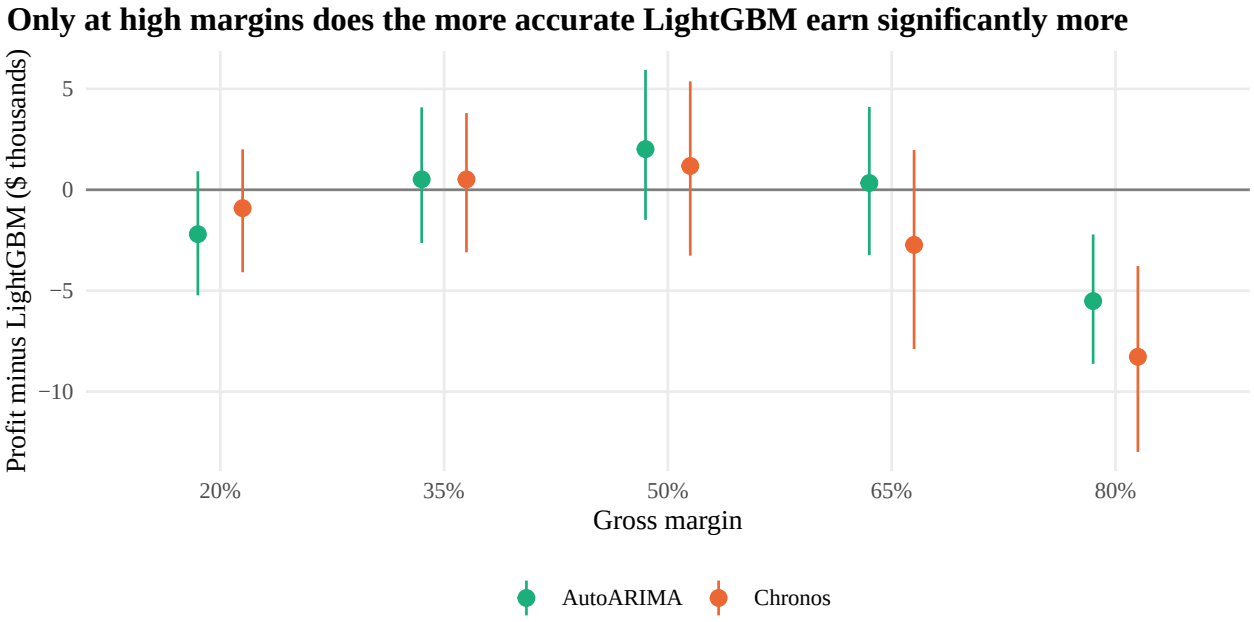


Figure 9: Profit difference against LightGBM over 28 days, with 95% bootstrap intervals over items. Intervals crossing zero mean no significant difference.

1. **The ordering rule matters more than the model.** Ordering the margin quantile instead of the forecast is worth up to $172k over 28 days; the choice of model, at most $20k.
2. **More accuracy did not reliably mean more profit.** At typical margins (35–65%), profit differences between models were within chance. Only at an 80% margin, where stock-outs are costly, did LightGBM earn significantly more.
3. **Accuracy is measured where orders are not placed.** LightGBM's accuracy lead is at aggregate levels, but stores order item by item, where the models are close. Models should be chosen on the metric that matches the decision.
4. **Profit expectations should come from backtests.** Both models over-predicted their own profit (at a 50% margin, LightGBM predicted $151k and earned $115k), because the simulation treats each forecast as the true average.

To explore these trade-offs interactively, see our dashboard, Shelf Signal, at https://jing0712.github.io/shelf-signal/#planner: it shows every model's forecasts for any series, and its order planner moves the order as the margin changes (Appendix A).

7 Conclusions

1. **Traditional methods still lead on accuracy.** LightGBM is the most accurate single model, AutoETS is close behind, and their simple average is the most accurate of all, a result our teammate's Shelf Signal study reproduces on a different setting.
2. **Rankings depend on the level and the window.** The models differ most at store and

department level, are close for single items, and close rankings can flip between test windows.

3. **AI wins on speed.** Chronos forecasts every series in 10 s with no training, against 39 min for AutoARIMA; Claude, the other AI, was slow and costly.
4. **Claude matched the best traditional approach on totals, on limited evidence.** It was level with the ensemble on the 12 totals and behind AutoETS on items, was tested on 112 series only, and contamination cannot be ruled out.
5. **Replenishment depends on costs and uncertainty.** The best order is a margin-dependent quantile of demand: ordering that quantile instead of the forecast was worth far more than switching models.

Practical recommendation: for item-level forecasts, average a statistical model (AutoETS) with LightGBM; use Chronos where speed or the lack of training data matters; fit store and department totals directly; and choose models on the metric that matches the decision they support.

8 Limitations and next steps

- **One test window.** Rankings between close models could change in another month, as Shelf Signal's validation and test windows show. A rolling-origin backtest over several earlier 28-day windows is the next step.
- **Generalisation.** All results come from one retailer, one category (food), four California stores and one month in 2016. Rankings may differ for other categories, regions, retailers or more recent data, and for newer or larger versions of the AI models.
- **Claude on a subset, with residual contamination risk.** Its results rest on 112 series. The blind test reduces identity cues but cannot guarantee the data was not seen in training, and its forecasts can vary between runs. A stronger check is a hidden test: data Claude cannot have seen, such as sales published after its training cutoff or a retailer's private data.
- **Claude is costly, and its explanations were not valued.** Every series is a paid request, so a daily run over thousands of items costs far more than the other models, which run free on a laptop. Claude also returns a written explanation with each forecast; we did not measure whether those explanations improve decisions enough to justify the cost.
- **Simple LightGBM.** One fixed setting; M5 winners tuned and ensembled many models.
- **Sales are not demand.** A stocked-out shelf records zero sales, so demand is understated for every model.
- **Newsvendor analysis is exploratory.** It assumed perishable stock, zero salvage value and, for every model except Chronos, a negative binomial demand distribution centred on the model's forecast, with each item's spread (variance-to-mean ratio) taken from its last 180 training days; using each model's own uncertainty (prediction intervals, quantile regression) would make it a fuller test. A promising next step is a hybrid: the ensemble's forecast for the level and Chronos's quantiles for the spread, scored by profit and pinball loss at

each margin.

References

Ansari, Abdul Fatir, Lorenzo Stella, Caner Turkmen, et al. 2024. "Chronos: Learning the Language of Time Series." *Transactions on Machine Learning Research*. https://arxiv.org/abs/2403.07815.

Makridakis, Spyros, Evangelos Spiliotis, and Vassilios Assimakopoulos. 2022. "M5 Accuracy Competition: Results, Findings, and Conclusions." *International Journal of Forecasting* 38 (4): 1346–64. https://doi.org/10.1016/j.ijforecast.2021.11.013.

University of Nicosia and Walmart. 2020. "M5 Forecasting – Accuracy." Kaggle competition. https://www.kaggle.com/competitions/m5-forecasting-accuracy.

Appendix A: Interactive dashboard (Shelf Signal)

We built **Shelf Signal**, an interactive version of this analysis, by extending our teammate's original Shelf Signal dashboard (code at https://github.com/Jing0712/shelf-signal) with this project's models and results. It is live at https://jing0712.github.io/shelf-signal/#planner (the link opens on the Order planner) and is a single self-contained web page (`docs/index.html` in the project repository) with seven tabs:

Tab	What it shows	Report section
Overview	Headline results, weekly and weekday sales patterns	1, 3
Forecast explorer	Any total, store, department, store × department or item; actual vs every model over the 28 test days, with per-series accuracy	5.1–5.2
Model comparison	Accuracy, accuracy by level, run time, and which model wins each item	5.1, 5.4
AI lab	Chronos vs Claude, the named-vs-blind test, and Claude's written reasoning for each series	5.3
Order planner	Choose an item, a model and a margin; the order moves along the forecast demand distribution, and profit over 28 days is scored for every model	6
Ask the analyst	Answers to 11 common questions, with numbers drawn from the results	All
About	Method, limitations and credits	8

It covers 134 series: every aggregate level, Claude's 100 sampled items, and the 15 store-CA_1 items from the original Shelf Signal. Forecasts are precomputed by our pipeline; only the order

quantities are calculated in the browser, and actual sales are used only to score results afterwards.

How to use it.

1. Open the link in any browser; nothing needs to be installed. The page also works offline from `docs/index.html`.
2. **Forecast explorer:** choose a level (for example *Store x dept*) and a series, then switch models on and off to compare them with actual sales.
3. **AI lab:** compare Chronos and Claude, see the named-vs-blind scatter, and open *Read Claude's reasoning* for any series.
4. **Order planner:** choose an item, a model and a margin; move the margin slider to see the order move along the demand distribution and how each model's orders would have done over the 28 test days.
5. **Ask the analyst:** click a question for a short answer drawn from the results.

A.1 Evidence from the original Shelf Signal

Shelf Signal studies a smaller, complementary setting: one store (CA_1), 15 high-selling food items plus the category total, and four models (seasonal naive, Holt-Winters ETS, gradient-boosted trees with Poisson loss, and their average), evaluated on a validation window and a final test window. Three of its results strengthen ours.

Table 10: Shelf Signal: item-level RMSSE (15 items, store CA_1) on the validation window and the final test window. Lower is better.

Model	Validation RMSSE	Test RMSSE
Ensemble	0.743	0.579
ETS	0.809	0.585
GBM	0.719	0.634
Seasonal Naive	0.958	0.778

1. **Rankings change between windows.** At item level, gradient-boosted trees beat ETS on the validation window (0.719 vs 0.809) but lost to it on the test window (0.634 vs 0.585). This is direct evidence for our main caveat: a single test window can reverse a close ranking.
2. **Averaging a statistical and a machine-learning model helps, in both studies.** Shelf Signal's ETS + GBM average had the lowest item-level test error (0.579), mirroring our AutoETS + LightGBM ensemble result on a different store, item set and model implementation.
3. **Holidays were not the hard part.** Scaled errors on event days (Super Bowl, Easter, Mother's Day and others) were 3%–6% *lower* than on normal days for every model.

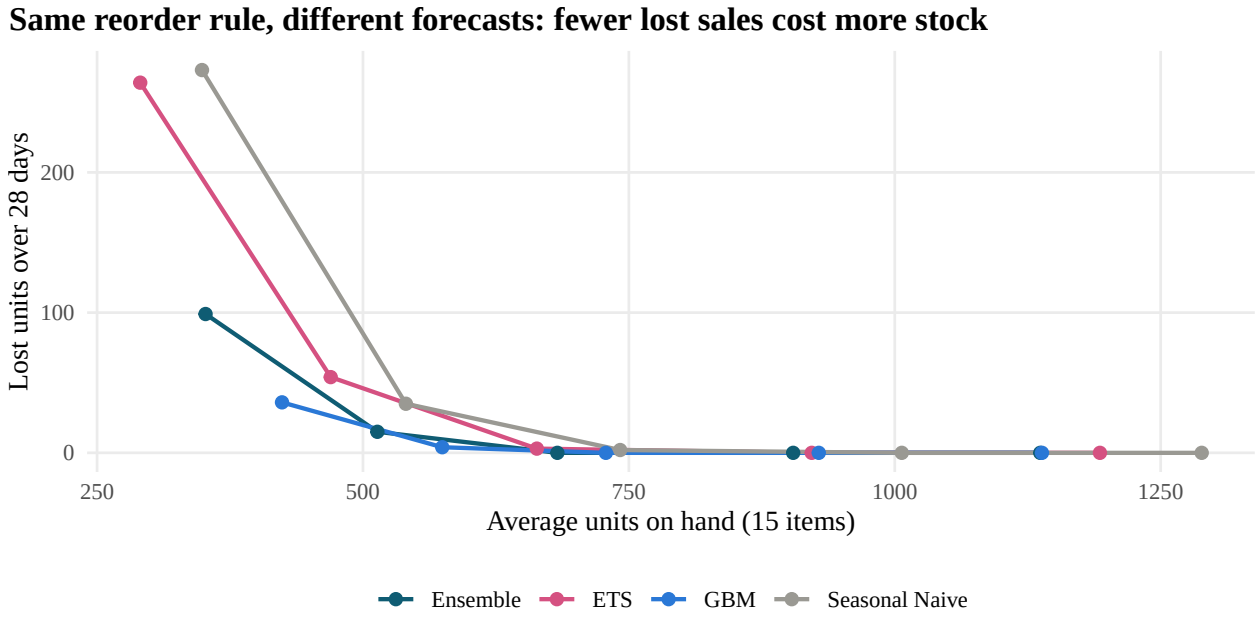


Figure 10: Shelf Signal inventory backtest (15 items, store CA_1, 28 days): raising the safety factor z cuts lost sales but raises the stock held. Each point is one model at one z.

The inventory backtest complements our newsvendor analysis. With no safety stock ($z = 0$), the forecasts lose very different amounts of demand (from 36 to 273 units); with enough safety stock, lost sales disappear for every model and the difference becomes how much stock each needs to hold. As in the newsvendor analysis, the choice of service level or margin moves the decision more than the choice of model. Shelf Signal notes that observed sales stand in for demand, so this is an illustration of the trade-off, not a cost estimate.

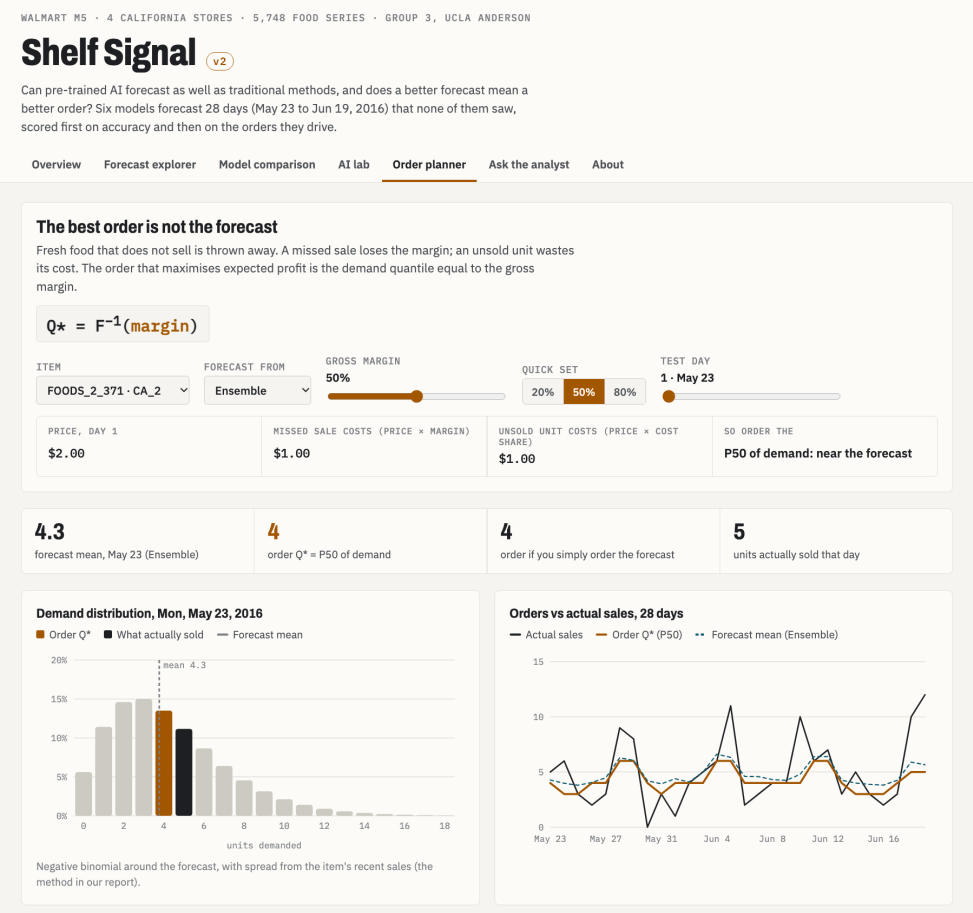


Figure 11: Shelf Signal, Order planner tab: the demand distribution for one item-day with the margin-matched order (orange) and actual sales (black), and orders against actual sales over the 28 test days. Live at https://jing0712.github.io/shelf-signal/#planner; built on Jing Chang's original Shelf Signal (https://github.com/Jing0712/shelf-signal).

Appendix B: Use of AI tools and reproducibility

AI played two roles in this project: as forecasting models under test, and as a coding and writing assistant.

Tool	What we used it for	How we checked it
Chronos-Bolt (small)	Zero-shot forecasts for all 5,748 series	Scored on the same held-out 28 days as every other model
Claude Opus 5.5 (API)	Zero-shot forecasts for 112 series ($13.54)	Fixed JSON format, 28-day check and retries; named vs blind test; every prompt and response saved in `outputs/claude_raw/`
Claude Code	Data pipeline, evaluation code, charts, dashboard, slides and report drafts	We reviewed all output and fixed three first-pass errors: LightGBM overfitting on the item ID, Chronos's median used as its mean, and an unfair summed-vs-direct comparison of totals. Every number in the report is read from the pipeline's output files

The forecasting pipeline is written in Python (`src/`); this report is written in R and reads its CSV outputs (`outputs/results/report/`). All code, results and the dashboard source are at https://github.com/shihan-2002/m5-forecast-to-profit, whose README lists the steps to reproduce everything.